

第3章 注意力机制——让模型学会"读懂"上下文

Ch02 走完了"文本 → token ID → embedding → 位置编码 → 输入张量"整条链路，数据准备好了。

本章回答的核心问题：**一堆 token 向量摆在那里，模型怎么知道哪些 token 之间有语义关联？**

开门见山：注意力机制就是大模型核心中的核心。

如果用一句话概括大模型在干什么，就是"分析一句话里每个 token 之间的关系，然后预测下一个 token"。

注意力机制负责前半句——**分析 token 之间的关系**。没有它，模型看到的就是一堆互不相干的词，根本搞不清"it"指的是"cat"还是"mat"。

这一章我们就来拆解这个机制到底是怎么算的。

大模型的训练目标很简单：

给定上文，猜下一个 token。比如看到 "I love"，应该猜出 "you"。模型内部靠什么猜？

靠的是理解 "I"、"love" 这些 token 之间的语法和语义关系——这恰恰是注意力机制要提供的。

所以，理解注意力，就是理解大模型怎么"看懂"一句话。

所以，我们先讨论一句话中，每个token之间的关系如何表示

不同token之间关系的表示

先导入需要的库，我们先用简单的例子看看

```
In [1]: from importlib.metadata import version
print("torch version:", version("torch"))
```

torch version: 2.12.1

从上一章，我们得到了带有位置编码的输入 token 向量。举个例子，假设 embedding 维度为 3，context_length=6：

输入是一句话 "Your journey starts with one step"，已经走完 Ch02 的 embedding 和位置编码流程，得到了 6 个 token、每个 3 维的向量矩阵。

这里的维度故意设得很小（3 维），是为了每一步都能把完整的矩阵打印出来。真实场景（256~1600 维）数值太多，不太方便看计算过程。

```
In [1]: import torch

inputs = torch.tensor(
    [[0.43, 0.15, 0.89], # Your
     [0.55, 0.87, 0.66], # journey
     [0.57, 0.85, 0.64], # starts
     [0.22, 0.58, 0.33], # with
     [0.05, 0.80, 0.55], # one
     [0.77, 0.25, 0.10]] # step
)

print("输入形状:", inputs.shape)
print("(6 个 token, 每个 3 维)\n")
print("输入矩阵:")
print(inputs)
```

输入形状: torch.Size([6, 3])
(6 个 token, 每个 3 维)

输入矩阵:

```
tensor([[0.4300, 0.1500, 0.8900],
        [0.5500, 0.8700, 0.6600],
        [0.5700, 0.8500, 0.6400],
        [0.2200, 0.5800, 0.3300],
        [0.0500, 0.8000, 0.5500],
        [0.7700, 0.2500, 0.1000]])
```

我们如何知道 Your 和 journey 这两个 token 的关系如何呢？

它们现在都用向量表示，而向量之间的关系可以通过点积（dot product）来计算出一个值：

$$\alpha \cdot \beta = |\alpha||\beta| \cos \theta$$

直观理解：两个向量方向越一致（夹角越小），点积越大；方向越正交，点积越接近 0。所以我们可以用点积表示两个 token 向量的“相关程度”。

严格来说，点积同时受方向和向量长度影响——很长的向量即使方向不太一致，点积也可能不小。后面会通过归一化手段来缓解这个问题。这里先按“方向相似度”来理解。

```
In [3]: # 计算Your 与 journey的点积
Your = inputs[0]
journey = inputs[1]

print("Your的矩阵表示: ", Your)
print("journey的矩阵表示: ", journey)

# 计算点积:
Your_journey_dot = torch.dot(Your, journey)
print("Your 与 journey的点积结果: ", Your_journey_dot)

# 手算验证一下
hand_result = 0
```

```

for i in range(3):
    hand_result = hand_result + Your[i] * journey[i]
print(
    f"torch.dot结果 = {Your_journey_dot},\n"
    f"手算结果 = {hand_result},\n"
    f"'与手算结果一致' if Your_journey_dot == hand_result else '与手算结果不一致'
)

```

Your的矩阵表示: `tensor([0.4300, 0.1500, 0.8900])`

journey的矩阵表示: `tensor([0.5500, 0.8700, 0.6600])`

Your 与 journey的点积结果: `tensor(0.9544)`

`torch.dot`结果 = `0.9544000625610352`,

手算结果 = `0.9544000625610352`,

与手算结果一致

既然如此, 我们可以这样计算所有token之间的点积

```

In [4]: dot = inputs @ inputs.T
print(dot)

```

```

tensor([[0.9995, 0.9544, 0.9422, 0.4753, 0.6310, 0.4576],
        [0.9544, 1.4950, 1.4754, 0.8434, 1.0865, 0.7070],
        [0.9422, 1.4754, 1.4570, 0.8296, 1.0605, 0.7154],
        [0.4753, 0.8434, 0.8296, 0.4937, 0.6565, 0.3474],
        [0.6310, 1.0865, 1.0605, 0.6565, 0.9450, 0.2935],
        [0.4576, 0.7070, 0.7154, 0.3474, 0.2935, 0.6654]])

```

这样, 只需要取Your序号行与 journey序号列, 即[0]行[1]列, 即可得到这两个token的向量点积结果, 我们可以看到dot矩阵的[0]行[1]列内容与我们刚刚手动计算的结果一致。

点积结果就是所谓的**注意力得分** (attention scores)。不过这些得分范围很大、不好直观比较, 所以我们把它归一化到 0~1 区间, 让每行加起来等于 1——这样就变成了一目了然的"关注度比例"。

最简单的方式: 把每个得分除以整行的总和。比如对于 Your (第 0 行), 把它跟所有 token 的点积得分分别除以这一行的总分:

```

In [5]: # dot[0] = Your 与其他所有 token 的点积结果 (6 个值)
# 除以它们的总和 → 归一化, 让结果加起来等于 1
norm_avg = dot[0] / torch.sum(dot[0])
print("Your 对各 token 的归一化关注度:")
print(norm_avg)
print(f"验证: 加起来 = {norm_avg.sum():.4f}")

```

Your 对各 token 的归一化关注度:

`tensor([0.2241, 0.2140, 0.2113, 0.1066, 0.1415, 0.1026])`

验证: 加起来 = `1.0000`

直接除总和虽然简单, 但实际项目中都用 **softmax**。原因有两个:

1. **放大差异**: softmax 的指数运算 (`exp`) 会让原本接近的值被拉得更开, 模型更容易"聚焦"到真正相关的 token, 而不是一碗水端平。
2. **梯度友好**: softmax 处处可导, 配合交叉熵损失函数时梯度计算特别简洁, 训练稳定。

下面先手写一个 naive 版 softmax 感受一下原理, 再换成 PyTorch 的优化版。

```
In [6]: # softmax 简单实现 — 仅用于理解原理
# 注意: 这个版本只支持 1D 张量 (比如 dot[0] 这种一行数据)
# PyTorch 的 torch.softmax 支持任意维度, 后面会用它
def softmax_naive(x):
    return torch.exp(x) / torch.exp(x).sum(dim=0) # dim=0 对 1D 张量没问题
```

```
In [7]: norm_softmax = softmax_naive(dot[0])
print(norm_softmax)

tensor([0.2098, 0.2006, 0.1981, 0.1242, 0.1452, 0.1220])
```

```
In [8]: # torch.softmax 对 2D 矩阵按行归一化
# dim=1 表示沿着列方向做 softmax — 每一行独立计算, 行内所有值加起来等于 1
# dot 的形状是 [6, 6], 6 个 token 各自对 6 个 token 的关注
attn_score = torch.softmax(dot, dim=1)
print("softmax 归一化后的注意力权重 (每行加起来 = 1): ")
print(attn_score)
print(f"\n验证每行和: {attn_score.sum(dim=1).tolist()}")
```

softmax 归一化后的注意力权重 (每行加起来 = 1):

```
tensor([[0.2098, 0.2006, 0.1981, 0.1242, 0.1452, 0.1220],
        [0.1385, 0.2379, 0.2333, 0.1240, 0.1581, 0.1082],
        [0.1390, 0.2369, 0.2326, 0.1242, 0.1565, 0.1108],
        [0.1435, 0.2074, 0.2046, 0.1462, 0.1720, 0.1263],
        [0.1385, 0.2184, 0.2128, 0.1420, 0.1896, 0.0988],
        [0.1526, 0.1958, 0.1975, 0.1367, 0.1295, 0.1879]])
```

验证每行和: [0.9999998807907104, 1.0, 1.0000001192092896, 1.0, 1.0, 0.9999999403953552]

```
In [9]: attn_score = torch.softmax(dot, dim=1) # dim=1: 按行归一化, 本身dot矩阵的维度是[6, 6]
print(attn_score)
```

```
tensor([[0.2098, 0.2006, 0.1981, 0.1242, 0.1452, 0.1220],
        [0.1385, 0.2379, 0.2333, 0.1240, 0.1581, 0.1082],
        [0.1390, 0.2369, 0.2326, 0.1242, 0.1565, 0.1108],
        [0.1435, 0.2074, 0.2046, 0.1462, 0.1720, 0.1263],
        [0.1385, 0.2184, 0.2128, 0.1420, 0.1896, 0.0988],
        [0.1526, 0.1958, 0.1975, 0.1367, 0.1295, 0.1879]])
```

有了注意力权重, 下一步就是按权重把所有 token 的信息汇总

——对每个 token, 用它对其他 token 的注意力权重, 对原始向量做加权求和, 得到"带上下文的新表示".

```
token 0 的新向量 = attn[0][0] × inputs[0]
                  + attn[0][1] × inputs[1]
                  + attn[0][2] × inputs[2]
                  + ...
```

矩阵形式就是一句话:

```
In [10]: # 加权求和: 注意力权重 @ 原始向量 → 带上下文的 token 表示
# attn_score: [6, 6] — 每行是 token i 对 6 个 token 的注意力权重
# inputs:      [6, 3] — 6 个 token, 每个 3 维
# context:     [6, 3] — 每个 token 得到了融合全句信息后的新向量
context = attn_score @ inputs
```

```

print("=== 加权求和：每个 token 的新向量 ===")
print(f"形状：{context.shape}（和原始 inputs 形状完全一样）\n")
for i in range(len(context)):
    print(f"token {i} 的新向量：{context[i].tolist()}")

print("\n对比：")
print(f"inputs[0]（原始）：{inputs[0].tolist()}")
print(f"context[0]（融合后）：{context[0].tolist()}")
print("→ 向量变了！它不再是 'Your' 独自的信息，而是融合了全句所有 token 的信息")

```

=== 加权求和：每个 token 的新向量 ===

形状：torch.Size([6, 3])（和原始 inputs 形状完全一样）

token 0 的新向量：[0.44205939769744873, 0.593098521232605, 0.5789890885353088]

token 1 的新向量：[0.4418657422065735, 0.6514819860458374, 0.5683088898658752]

token 2 的新向量：[0.4431275427341461, 0.6495946049690247, 0.5670731067657471]

token 3 的新向量：[0.43038973212242126, 0.6298280954360962, 0.5510269999504089]

token 4 的新向量：[0.41772449016571045, 0.6503232717514038, 0.5645352602005005]

token 5 的新向量：[0.46710169315338135, 0.5909926891326904, 0.5265965461730957]

对比：

inputs[0]（原始）：[0.4300000071525574, 0.15000000596046448, 0.8899999856948853]

context[0]（融合后）：[0.44205939769744873, 0.593098521232605, 0.5789890885353088]

→ 向量变了！它不再是 'Your' 独自的信息，而是融合了全句所有 token 的信息

Part 1 小结：我们做了什么、还差什么

到这里，你已经看到了注意力机制的**核心计算链**：

原始向量 $\rightarrow$ 点积(算相似度) $\rightarrow$ softmax(归一化成权重) $\rightarrow$ 加权求和(融合上下文)

[6,3]

[6,6]

[6,6]

[6,3]

每个 token 的向量不再是孤立的——它融合了整句话里所有 token 的信息，融合比例由注意力权重决定。

但这个方案有一个根本缺陷：****注意力权重完全由输入的原始 embedding 决定，没有任何可训练的参数。**

**** 模型无法学会“当我是动词时该多看主语”“当我是代词时该找前面的名词”——它的注意力模式被初始 embedding 锁死了。**

下一节 (Part 2) 引入 **Q/K/V 三个可训练矩阵**，把“去问”“被问”“传信息”这三件事拆开，让模型自己学会怎么分配注意力。这也就是 Transformer 论文里真正的 **scaled dot-product self-attention**。